

DATA 505: Lab 2

Getting to know data frames

YOUR NAME HERE

2025-09-02

Table of contents

1	Setup & Goals	1
2	The data file	2
3	Reading data into R	2
4	Subsetting the data	3
5	Exploring column types and coercion	4
6	Bonus	5

1 Setup & Goals

Just like in [Lab 1](#), you will edit a `.qmd` Quarto script, render the result to a PDF, and submit both on Moodle (if you do not succeed at rendering the PDF, still submit the `.qmd` script: you may still get full “satisfactory” credit if you have made a reasonable effort).

Goals for Lab 2:

- Gain more fluency with writing R code in the context of a Quarto script in RStudio
- Learn to read data into R from a text (comma-separated values, `.csv`) file
- Reinforce concepts from [this week’s lecture](#) about types and classes of variables
- Get some practice taking subsets of data frame objects

2 The data file

To do this, we will use the same dataset as in [Section 7](#) of our primary textbook

- The following code will download the `students.csv` dataset to your current working directory. You should leave this code unmodified:

```
download.file("https://andrew-bean.github.io/data505/labs/week_2/students.csv", "students.csv",  
stopifnot(file.exists("students.csv"))
```

- I recommend running this code above first before writing any more code (with your cursor in the code chunk, from the “Run” menu, you can select “Run current chunk”)

3 Reading data into R

- Take a look at the file you have just downloaded. It should look like [this](#).
- The base R function for reading `.csv` files into data frames is `read.csv()`. An example usage would be

```
# just an example: no need to edit  
my_data <- read.csv("my_file.csv")
```

- Try using `read.csv` to read the `students.csv` file and store the result in an object named `students`:

```
students <- read.csv("students.csv")
```

- Write a line of code to determine the **class** of the resulting `students` object

```
class(students)
```

```
[1] "data.frame"
```

- Write another line of code to determine the **basic type** of the `students` object

```
typeof(students)
```

```
[1] "list"
```

- Apply the `str()` function to the `students` object to learn more about its structure

```
str(students)
```

```
'data.frame':  6 obs. of  5 variables:
 $ Student.ID      : int  1 2 3 4 5 6
 $ Full.Name       : chr  "Sunil Huffmann" "Barclay Lynn" "Jayendra Lyne" "Leon Rossini" ...
 $ favourite.food  : chr  "Strawberry yoghurt" "French fries" "N/A" "Anchovies" ...
 $ mealPlan        : chr  "Lunch only" "Lunch only" "Breakfast and lunch" "Lunch only" ...
 $ AGE             : chr  "4" "5" "7" "N/A" ...
```

- Now use the `names()` function to get a list of the column names

```
names(students)
```

```
[1] "Student.ID"      "Full.Name"      "favourite.food" "mealPlan"
[5] "AGE"
```

4 Subsetting the data

- Use `$` (as discussed [here in today's lecture](#)) to select the `AGE` column and store it as another object named `ages`

```
ages <- students$AGE
```

- Now repeat using `[["{column name}"]]` syntax illustrated at the same link

```
ages <- students[["AGE"]]
```

- Use the `length()` function to determine the length of the `ages` vector

```
length(ages)
```

```
[1] 6
```

- Define another variable `age` which consists only of the first element of the `ages` vector. Recall we discussed vector subsetting [here in today's lecture](#).

```
age <- ages[1]
```

- Now something a bit more challenging. Use data frame subsetting, a [relational operator](#) and vector subsetting together to do the following: return a vector containing the ages of all students on the “Lunch only” meal plan

```
students$AGE[students$mealPlan == "Lunch only"]
```

```
[1] "4"    "5"    "N/A" "6"
```

5 Exploring column types and coercion

- Write a line of code to determine the type of the `ages` vector you created earlier (the AGE column from the `students` data frame)

```
typeof(ages)
```

```
[1] "character"
```

- This variable ought to be numeric. Use [explicit coercion](#) to convert it.

```
ages <- as.numeric(ages)
```

Warning: NAs introduced by coercion

- Modify the `students` dataset to ensure the AGE column is numeric

```
students$AGE <- ages
```

- What is the type of the `mealPlan` column? Write a line of code to determine this.

```
typeof(students$mealPlan)
```

```
[1] "character"
```

- Modify the `students` dataset to ensure the `mealPlan` column is a factor

```
students$mealPlan <- as.factor(students$mealPlan)
```

6 Bonus

What we have done here is some cumbersome “data cleaning”. We read data in from a file, but `read.csv` did not recognize the intended types of the csv columns immediately. So we coerced some columns from characters to factors or numerics.

- Try following some of [Section 7.2.1](#) of the R for Data Science book to try to do this more concisely with the `read_csv()` function in the `readr` package.

```
# install the readr package if needed
if(!"readr" %in% installed.packages()){
  install.packages("readr")
}
# load it
library(readr)

students <- read_csv("students.csv",
                     na = c("N/A", ""),
                     col_types = list(mealPlan = col_factor(),
                                       AGE = col_double()))

str(students)
```

```
spc_tbl_ [6 x 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ Student ID      : num [1:6] 1 2 3 4 5 6
 $ Full Name       : chr [1:6] "Sunil Huffmann" "Barclay Lynn" "Jayendra Lyne" "Leon Rossini"
 $ favourite.food  : chr [1:6] "Strawberry yoghurt" "French fries" NA "Anchovies" ...
 $ mealPlan        : Factor w/ 2 levels "Lunch only","Breakfast and lunch": 1 1 2 1 2 1
 $ AGE             : num [1:6] 4 5 7 NA 5 6
 - attr(*, "spec")=
  .. cols(
  ..   `Student ID` = col_double(),
  ..   `Full Name` = col_character(),
  ..   favourite.food = col_character(),
  ..   mealPlan = col_factor(levels = NULL, ordered = FALSE, include_na = FALSE),
  ..   AGE = col_double()
  .. )
 - attr(*, "problems")=<externalptr>
```

```
file.remove("students.csv") # don't need this file anymore
```

```
[1] TRUE
```